8 Разработка базы данных на основе разработанной ER-модели

1. Используя результаты работы из практики №1 (модель базы данных), разработать базу данных в СУБД PostgreSQL.

```
CREATE TABLE countries (
    country_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    continent VARCHAR(255) NOT NULL,
    CONSTRAINT unique_country_name UNIQUE (name)
);
```

```
postgres=# CREATE TABLE countries (
    country_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    continent VARCHAR(255) NOT NULL,
    CONSTRAINT unique_country_name UNIQUE (name)
);
CREATE TABLE
```

```
CREATE TABLE climate_zones (
    climate_zone_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    avg_temperature DECIMAL(5,2),
    humidity DECIMAL(5,2) CHECK (humidity BETWEEN 0 AND 100),
    seasonality VARCHAR(255),
    precipitation DECIMAL(10,2),
    features TEXT,
    CONSTRAINT unique_climate_name UNIQUE (name)
);
```

```
postgres=# CREATE TABLE climate_zones (
    climate_zone_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    avg_temperature DECIMAL(5,2),
    humidity DECIMAL(5,2) CHECK (humidity BETWEEN 0 AND 100),
    seasonality VARCHAR(255),
    precipitation DECIMAL(5,2),
    features TEXT,
    CONSTRAINT unique_climate_name UNIQUE (name)
);
CREATE TABLE
```

```
CREATE TABLE hotels (
    hotel_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    stars DECIMAL(3,2) CHECK (stars BETWEEN 0 AND 5),
    address TEXT NOT NULL,
    CONSTRAINT unique_hotel_name UNIQUE (name)
);
```

```
postgres=# CREATE TABLE hotels (
    hotel_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    stars DECIMAL(3,2) CHECK (stars BETWEEN 0 AND 5),
    address TEXT NOT NULL,
    CONSTRAINT unique_hotel_name UNIQUE (name)
);
CREATE TABLE
```

```
CREATE TABLE routes (
    route_id INT PRIMARY KEY,
    country_id INT NOT NULL REFERENCES countries(country_id) ON DELETE SET
DEFAULT,
    climate_zone_id INT NOT NULL REFERENCES climate_zones(climate_zone_id)
ON DELETE SET DEFAULT,
    duration DATE NOT NULL,
    cost DECIMAL(10,2) CHECK (cost > 0),
    departure_address VARCHAR(255) NOT NULL,
    CONSTRAINT unique_route_departure UNIQUE (departure_address)
);
```

```
postgres=# CREATE TABLE routes (
    route_id INT PRIMARY KEY,
    country_id INT NOT NULL REFERENCES countries(country_id) ON DELETE SET DEFAULT,
    climate_zone_id INT NOT NULL REFERENCES climate_zones(climate_zone_id) ON DELETE SET DEFAULT,
    duration DATE NOT NULL,
    cost DECIMAL(10,2) CHECK (cost > 0),
    departure_address VARCHAR(255) NOT NULL,
    CONSTRAINT unique_route_departure UNIQUE (departure_address)
);
CREATE TABLE
```

```
CREATE TABLE route_hotels (
    route_id INT NOT NULL REFERENCES routes(route_id) ON DELETE CASCADE,
    hotel_id INT NOT NULL REFERENCES hotels(hotel_id) ON DELETE CASCADE,
    PRIMARY KEY (route_id, hotel_id)
);
```

```
postgres=# CREATE TABLE route_hotels (
    route_id INT NOT NULL REFERENCES routes(route_id) ON DELETE CASCADE,
    hotel_id INT NOT NULL REFERENCES hotels(hotel_id) ON DELETE CASCADE,
    PRIMARY KEY (route_id, hotel_id)
);
CREATE TABLE
```

```
CREATE TABLE clients (
    client_id INT PRIMARY KEY,
    surname VARCHAR(255) NOT NULL,
    name VARCHAR(255) NOT NULL,
    patronymic VARCHAR(255),
    address TEXT,
    phone VARCHAR(31),
    CONSTRAINT unique_client_phone UNIQUE (phone)
);
```

```
postgres=# CREATE TABLE clients (
    client_id INT PRIMARY KEY,
    surname VARCHAR(255) NOT NULL,
    name VARCHAR(255) NOT NULL,
    patronymic VARCHAR(255),
    address TEXT,
    phone VARCHAR(31),
    CONSTRAINT unique_client_phone UNIQUE (phone)
);
CREATE TABLE
```

```
CREATE TABLE vouchers (
    voucher_id INT PRIMARY KEY,
    route_id INT NOT NULL REFERENCES routes(route_id) ON DELETE SET
```

```
 DEFAULT,
     client_id INT NOT NULL REFERENCES clients(client_id) ON DELETE SET
 DEFAULT,
     hotel_id INT NOT NULL REFERENCES hotels(hotel_id) ON DELETE SET
 DEFAULT,
     departure_date DATE NOT NULL,
     people_count INT CHECK (people_count > 0),
     discount DECIMAL(5,2) CHECK (discount >= 0)
 );
```

```
postgres=# CREATE TABLE vouchers (
    voucher_id INT PRIMARY KEY,
    route_id INT NOT NULL REFERENCES routes(route_id) ON DELETE SET DEFAULT,
    client_id INT NOT NULL REFERENCES clients(client_id) ON DELETE SET DEFAULT,
    hotel_id INT NOT NULL REFERENCES hotels(hotel_id) ON DELETE SET DEFAULT,
    departure_date DATE NOT NULL,
    people_count INT CHECK (people_count > 0),
    discount DECIMAL(5,2) CHECK (discount >= 0)
);
CREATE TABLE
```

```
 CREATE TABLE discount_cards (
     card_id INT PRIMARY KEY,
     amount DECIMAL(10,2) CHECK (amount > 0),
     expiration_date DATE,
     client_id INT NOT NULL REFERENCES clients(client_id) ON DELETE SET
 DEFAULT
 );
```

```
postgres=# CREATE TABLE discount_cards (
    card_id INT PRIMARY KEY,
    amount DECIMAL(10,2) CHECK (amount > 0),
    expiration_date DATE,
    client_id INT NOT NULL REFERENCES clients(client_id) ON DELETE SET DEFAULT
);
CREATE TABLE
```

```
 CREATE TABLE discounts (
     discount_id INT PRIMARY KEY,
     description TEXT,
     percent_discount DECIMAL(5,2) CHECK (percent_discount BETWEEN 0 AND
 100),
     min_quantity INT CHECK (min_quantity > 0),
     is_cumulative BOOLEAN NOT NULL,
     start_date DATE NOT NULL,
     end_date DATE
 );
```

```
postgres=# CREATE TABLE discounts (
    discount_id INT PRIMARY KEY,
    description TEXT,
    percent_discount DECIMAL(5,2) CHECK (percent_discount BETWEEN 0 AND 100),
    min_quantity INT CHECK (min_quantity > 0),
    is_cumulative BOOLEAN NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE
);
CREATE TABLE
```

2. Заполнить таблицы данными в соответствии со структурой.

```
INSERT INTO countries VALUES
(1, 'Таиланд', 'Азия'),
(2, 'Франция', 'Европа'),
(3, 'Испания', 'Европа'),
(4, 'Италия', 'Европа');
```

```
postgres=# INSERT INTO countries VALUES
(1, 'Таиланд', 'Азия'),
(2, 'Франция', 'Европа'),
(3, 'Испания', 'Европа');
INSERT 0 3
```

```
postgres=# INSERT INTO countries VALUES
(4, 'Италия', 'Европа');
INSERT 0 1
postgres=#
```

```
INSERT INTO climate_zones VALUES
(1, 'Тропический', 'Жаркий и влажный', 28.5, 75, 'Сезон дождей', 2000,
'Муссонный'),
(2, 'Умеренный', 'Теплое лето, мягкая зима', 15.0, 60, 'Без выраженных
сезонов', 800, '');
```

```
postgres=# INSERT INTO climate_zones VALUES
(1, 'Тропический', 'Жаркий и влажный', 28.5, 75, 'Сезон дождей', 2000, 'Муссонный'),
(2, 'Умеренный', 'Теплое лето, мягкая зима', 15.0, 60, 'Без выраженных сезонов', 800, '');
INSERT 0 2
postgres=#
```

```
INSERT INTO hotels VALUES
(1, 'Палм Бич', 4.5, 'ул. Солнечная, 10'),
```

```
(2, 'Ривьера', 5.0, 'ул. Королевская, 22');
```

```
postgres=# INSERT INTO hotels VALUES
(1, 'Палм Бич', 4.5, 'ул. Солнечная, 10'),
(2, 'Ривьера', 5.0, 'ул. Королевская, 22');
INSERT 0 2
postgres=#
```

```
INSERT INTO routes VALUES
(1, 1, 1, '2025-07-01', 1000.00, 'г. Москва, ул. Центральная 1'),
(2, 2, 2, '2025-06-15', 1200.00, 'г. Санкт-Петербург, пл. Восстания');
```

```
postgres=# INSERT INTO routes VALUES
(1, 1, 1, '2025-07-01', 1000.00, 'г. Москва, ул. Центральная 1'),
(2, 2, 2, '2025-06-15', 1200.00, 'г. Санкт-Петербург, пл. Восстания');
INSERT 0 2
postgres=#
```

```
INSERT INTO route_hotels VALUES
(1, 1),
(2, 2);
```

```
postgres=# INSERT INTO route_hotels VALUES
(1, 1),
(2, 2);
INSERT 0 2
postgres=#
```

```
INSERT INTO clients VALUES
(1, 'Иванов', 'Иван', 'Иванович', 'ул. Ленина, 1', '+79001234567'),
(2, 'Петров', 'Петр', 'Петрович', 'ул. Гагарина, 10', '+79001234568'),
(3, 'Сидоров', 'Сергей', 'Сергеевич', 'ул. Мира, 5', '+79001234569'),
(4, 'Кузнецов', 'Алексей', 'Алексеевич', 'ул. Пушкина, 15',
'+79001234570');
```

```
postgres=# INSERT INTO clients VALUES
(1, 'Иванов', 'Иван', 'Иванович', 'ул. Ленина, 1', '+79001234567'),
(2, 'Петров', 'Петр', 'Петрович', 'ул. Гагарина, 10', '+79001234568')
INSERT 0 2
postgres=#
```

```
postgres=# INSERT INTO clients VALUES
(3, 'Сидоров', 'Сергей', 'Сергеевич', 'ул. Мира, 5', '+79001234569'),
(4, 'Кузнецов', 'Алексей', 'Алексеевич', 'ул. Пушкина, 15', '+79001234570');
INSERT 0 2
postgres=#
```

```sql
INSERT INTO vouchers VALUES
(1, 1, 1, 1, '2025-07-01', 2, 10.00),
(2, 2, 2, 2, '2025-06-15', 1, 5.00),
(3, 1, 1, 1, '2025-08-01', 2, 10.00),
(4, 2, 3, 2, '2025-07-15', 1, 5.00);
```

```
postgres=# INSERT INTO vouchers VALUES
(1, 1, 1, 1, '2025-07-01', 2, 10.00),
(2, 2, 2, 2, '2025-06-15', 1, 5.00);
INSERT 0 2
postgres=#
```

```
postgres=# INSERT INTO vouchers VALUES
(3, 1, 1, 1, '2025-08-01', 2, 10.00);
INSERT 0 1
postgres=#
```

```
postgres=# INSERT INTO vouchers VALUES
(4, 2, 3, 2, '2025-07-15', 1, 5.00);
INSERT 0 1
postgres=#
```

```sql
INSERT INTO discount_cards VALUES
(1, 500.00, '2025-12-31', 1),
(2, 1000.00, '2026-01-01', 2),
(3, 700.00, '2025-11-30', 3);
```

```
postgres=# INSERT INTO discount_cards VALUES
(1, 500.00, '2025-12-31', 1),
(2, 1000.00, '2026-01-01', 2);
INSERT 0 2
postgres=#
```

```
postgres=# INSERT INTO discount_cards VALUES
(3, 700.00, '2025-11-30', 3);
INSERT 0 1
postgres=#
```

```sql
INSERT INTO discounts VALUES
(1, 'Скидка за покупку более одной путевки', 5.00, 2, TRUE, '2025-01-01',
'2025-12-31'),
(2, 'Сезонная распродажа', 10.00, 1, FALSE, '2025-06-01', '2025-08-31');
```

```
postgres=# INSERT INTO discounts VALUES
(1, 'Скидка за покупку более одной путевки', 5.00, 2, TRUE, '2025-01-01', '2025-12-31'),
(2, 'Сезонная распродажа', 10.00, 1, FALSE, '2025-06-01', '2025-08-31');
INSERT 0 2
postgres=#
```

3. Выполнить запросы к базе данных (выборка данных должна быть из не менее двух таблиц, для соединения использовать JOIN)

Количество клиентов по странам

```sql
SELECT c.name AS country, COUNT(cl.client_id) AS client_count
FROM clients cl
JOIN vouchers v ON cl.client_id = v.client_id
JOIN routes r ON v.route_id = r.route_id
JOIN countries c ON r.country_id = c.country_id
GROUP BY c.name
HAVING COUNT(cl.client_id) > 1;
```

```
postgres=# SELECT c.name AS country, COUNT(cl.client_id) AS client_count
FROM clients cl
JOIN vouchers v ON cl.client_id = v.client_id
JOIN routes r ON v.route_id = r.route_id
JOIN countries c ON r.country_id = c.country_id
GROUP BY c.name
HAVING COUNT(cl.client_id) > 1;
 country | client_count
---------+--------------
 Таиланд |            2
 Франция |            2
(2 rows)

postgres=#
```

Средняя стоимость тура по климатическим зонам

```sql
SELECT cz.name AS climate_zone, AVG(r.cost) AS avg_cost
FROM routes r
JOIN climate_zones cz ON r.climate_zone_id = cz.climate_zone_id
GROUP BY cz.name
HAVING AVG(r.cost) > 900;
```

```
postgres=# SELECT cz.name AS climate_zone, AVG(r.cost) AS avg_cost
FROM routes r
JOIN climate_zones cz ON r.climate_zone_id = cz.climate_zone_id
GROUP BY cz.name
HAVING AVG(r.cost) > 900;
 climate_zone |       avg_cost
--------------+----------------------
 Умеренный    | 1200.0000000000000000
 Тропический  | 1000.0000000000000000
(2 rows)
```

Клиенты с более чем одной путевкой

```sql
SELECT cl.surname, cl.name, COUNT(v.voucher_id) AS voucher_count
FROM clients cl
JOIN vouchers v ON cl.client_id = v.client_id
GROUP BY cl.surname, cl.name
HAVING COUNT(v.voucher_id) > 1;
```

```
postgres=# SELECT cl.surname, cl.name, COUNT(v.voucher_id) AS voucher_count
FROM clients cl
JOIN vouchers v ON cl.client_id = v.client_id
GROUP BY cl.surname, cl.name
HAVING COUNT(v.voucher_id) > 1;
 surname | name | voucher_count
---------+------+---------------
 Иванов  | Иван |             2
(1 row)
```

Некоррелированный: Клиенты без дисконтной карты

```sql
SELECT surname, name
FROM clients
WHERE client_id NOT IN (SELECT DISTINCT client_id FROM discount_cards);
```

```
postgres=# SELECT surname, name
FROM clients
WHERE client_id NOT IN (SELECT DISTINCT client_id FROM discount_cards);
 surname  |  name
----------+--------
 Кузнецов | Алексей
(1 row)
```

Коррелированный: Клиенты с максимальной суммой на карте

```sql
SELECT surname, name
FROM clients cl
WHERE EXISTS (
    SELECT 1
    FROM discount_cards dc
    WHERE dc.client_id = cl.client_id
    ORDER BY amount DESC LIMIT 1
);
```

```
postgres=# SELECT surname, name
FROM clients cl
WHERE EXISTS (
    SELECT 1
    FROM discount_cards dc
    WHERE dc.client_id = cl.client_id
    ORDER BY amount DESC LIMIT 1
);
 surname | name
---------+------
 Петров  | Петр
 Иванов  | Иван
(2 rows)
```

ROLLUP по странам и отелям

```sql
SELECT c.name AS country, h.name AS hotel, SUM(v.people_count) AS
total_people
FROM vouchers v
JOIN routes r ON v.route_id = r.route_id
JOIN countries c ON r.country_id = c.country_id
JOIN hotels h ON v.hotel_id = h.hotel_id
GROUP BY ROLLUP(c.name, h.name);
```

```
postgres=# SELECT c.name AS country, h.name AS hotel, SUM(v.people_count) AS total_people
FROM vouchers v
JOIN routes r ON v.route_id = r.route_id
JOIN countries c ON r.country_id = c.country_id
JOIN hotels h ON v.hotel_id = h.hotel_id
GROUP BY ROLLUP(c.name, h.name);
 country |  hotel   | total_people
---------+----------+--------------
         |          |            3
 Таиланд | Палм Бич |            2
 Франция | Ривьера  |            1
 Таиланд |          |            2
 Франция |          |            1
(5 rows)
```

GROUPING SETS: по странам и климатическим зонам

```sql
SELECT country_id, climate_zone_id, SUM(cost)
FROM routes
GROUP BY GROUPING SETS ((country_id), (climate_zone_id));
```

```
postgres=# SELECT country_id, climate_zone_id, SUM(cost)
FROM routes
GROUP BY GROUPING SETS ((country_id), (climate_zone_id));
 country_id | climate_zone_id |   sum
------------+-----------------+---------
          2 |                 | 1200.00
          1 |                 | 1000.00
            |               2 | 1200.00
            |               1 | 1000.00
(4 rows)
```

CUBE: комбинации стран и отелей

```sql
SELECT c.name AS country, h.name AS hotel, SUM(v.people_count) AS
total_people
FROM vouchers v
JOIN routes r ON v.route_id = r.route_id
JOIN countries c ON r.country_id = c.country_id
JOIN hotels h ON v.hotel_id = h.hotel_id
GROUP BY CUBE(c.name, h.name);
```

```
postgres=# SELECT c.name AS country, h.name AS hotel, SUM(v.people_count) AS total_people
FROM vouchers v
JOIN routes r ON v.route_id = r.route_id
JOIN countries c ON r.country_id = c.country_id
JOIN hotels h ON v.hotel_id = h.hotel_id
GROUP BY CUBE(c.name, h.name);
 country |  hotel   | total_people
---------+----------+--------------
         |          |            3
 Таиланд | Палм Бич |            2
 Франция | Ривьера  |            1
 Таиланд |          |            2
 Франция |          |            1
         | Ривьера  |            1
         | Палм Бич |            2
(7 rows)
```

LEAD — следующая путевка клиента

```sql
SELECT client_id, voucher_id, departure_date,
       LEAD(departure_date) OVER (PARTITION BY client_id ORDER BY
departure_date) AS next_trip
FROM vouchers;
```

```
postgres=# SELECT client_id, voucher_id, departure_date,
      LEAD(departure_date) OVER (PARTITION BY client_id ORDER BY departure_date) AS next_trip
FROM vouchers;
 client_id | voucher_id | departure_date | next_trip
-----------+------------+----------------+-----------
         1 |          1 | 2025-07-01     |
         2 |          2 | 2025-06-15     |
(2 rows)
```

LAG — предыдущая путевка клиента

```
SELECT client_id, voucher_id, departure_date,
       LAG(departure_date) OVER (PARTITION BY client_id ORDER BY
departure_date) AS prev_trip
FROM vouchers;
```

```
postgres=# SELECT client_id, voucher_id, departure_date,
      LAG(departure_date) OVER (PARTITION BY client_id ORDER BY departure_date) AS prev_trip
FROM vouchers;
 client_id | voucher_id | departure_date | prev_trip
-----------+------------+----------------+-----------
         1 |          1 | 2025-07-01     |
         2 |          2 | 2025-06-15     |
(2 rows)
```

LEAD + процент скидки

```
SELECT d.discount_id, d.percent_discount,
       LEAD(d.percent_discount) OVER (ORDER BY d.discount_id) AS
next_discount
FROM discounts d;
```

```
postgres=# SELECT d.discount_id, d.percent_discount,
       LEAD(d.percent_discount) OVER (ORDER BY d.discount_id) AS next_discount
FROM discounts d;
 discount_id | percent_discount | next_discount
-------------+------------------+---------------
           1 |             5.00 |         10.00
           2 |            10.00 |
(2 rows)
```

Категоризация отелей по количеству звезд

```
SELECT name, stars,
       CASE
           WHEN stars >= 4 THEN 'Высокий уровень'
           WHEN stars >= 3 THEN 'Средний уровень'
           ELSE 'Низкий уровень'
       END AS category
FROM hotels;
```

```
postgres=# SELECT name, stars,
       CASE
           WHEN stars >= 4 THEN 'Высокий уровень'
           WHEN stars >= 3 THEN 'Средний уровень'
           ELSE 'Низкий уровень'
       END AS category
FROM hotels;
   name    | stars |    category
-----------+-------+-----------------
 Палм Бич |  4.50 | Высокий уровень
 Ривьера  |  5.00 | Высокий уровень
(2 rows)
```

Использование CTE для подсчета средней стоимости путевок по странам

```sql
WITH AvgCostByCountry AS (
    SELECT c.name AS country, AVG(r.cost) AS avg_cost
    FROM routes r
    JOIN countries c ON r.country_id = c.country_id
    GROUP BY c.name
)
SELECT * FROM AvgCostByCountry WHERE avg_cost > 1000;
```

```
postgres=# WITH AvgCostByCountry AS (
    SELECT c.name AS country, AVG(r.cost) AS avg_cost
    FROM routes r
    JOIN countries c ON r.country_id = c.country_id
    GROUP BY c.name
)
SELECT * FROM AvgCostByCountry WHERE avg_cost > 1000;
 country |       avg_cost
---------+----------------------
 Франция | 1200.000000000000000
(1 row)
```